Table des matières

Exercices corrigés 07 — Sous-requêtes, corrélation et ensembles 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 5
Corrigé 6

Exercices corrigés 07 — Sous-requêtes, corrélation et ensembles

Comment travailler ces exercices. Répondez à chaque énoncé seul, puis vérifiez. Les questions courtes vérifient que le vocabulaire est en place ; les pratiques vous font écrire du SQL, tantôt sur le transactionnel, tantôt sur l'entrepôt ; les diagnostics vous montrent du code qui cloche ; les mises en situation vous demandent de choisir une approche et de la défendre. Comptez trois heures et trois quarts pour y répondre, le corrigé non compris.

Les repères de terrain. Un énoncé marqué `[OLTP]` se traite dans PostgreSQL sur `Adventureworks` (schémas `sales`, `production`, `person`). Un énoncé marqué `[OLAP]` se traite dans DuckDB sur `adventureworksdw.duckdb`, ouvert par `USE dbo;`. Sans marque, la question vaut pour les deux.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer : c'est en butant sur une question qu'on découvre ce qu'on avait lu sans le comprendre.

Questions courtes

1. Le manque du `GROUP BY`. Donnez deux questions métier qu'une agrégation ordinaire ne sait pas exprimer, et dites en une phrase ce qui leur manque à toutes les deux.
2. Les trois formes. Nommez les trois formes de sous-requête et, pour chacune, l'endroit de la requête où elle a le droit d'apparaître.
3. Une sous-requête scalaire placée dans un `WHERE` ne renvoie aucune ligne. Que vaut-elle, et qu'arrive-t-il à la ligne testée ?
4. Une sous-requête scalaire renvoie deux lignes au lieu d'une. Que se passe-t-il, et en quoi est-ce moins dangereux que le cas de la question 3 ?
5. Corrélée ou pas. Comment reconnaît-on, à la seule lecture, qu'une sous-requête est corrélée ?

© 2026 iAlexMG. Tous droits réservés.

6. « Une sous-requête corrélée est lente, il faut toujours la réécrire en jointure. » Cette phrase était vraie autrefois. Pourquoi ne l'est-elle plus, et que faut-il faire à la place de la croire ?
7. Dans un `EXISTS`, on écrit habituellement `SELECT 1`. Écrire `SELECT *` changerait-il le résultat ? Pourquoi cette convention alors ?
8. Semi-jointure et anti-jointure. Définissez chacune en une phrase, et donnez le mot-clé SQL qui la porte.
9. Pourquoi `EXISTS` est-il en général moins coûteux qu'un `COUNT(*) > 0` équivalent ?
10. Le piège central du module. Expliquez, sans citer de table réelle, pourquoi `x NOT IN (SELECT c FROM t)` ne renvoie aucune ligne dès que `c` contient un seul `NULL`.
11. Toujours sur ce piège : pourquoi est-il plus dangereux qu'une erreur de syntaxe ?
12. Trois écritures existent pour une anti-jointure. Citez-les, et dites laquelle vous écrivez par défaut.
13. Équivalences. Récrivez `x IN (…)` et `x NOT IN (…)` avec `ANY` et `ALL`.
14. Que vaut `x > ALL (…)` quand la sous-requête ne renvoie aucune ligne ? Et `x > ANY (…)` dans le même cas ? (avancé)
15. Les quatre opérateurs d'ensembles. Citez-les et dites lequel ne dédoublonne pas.
16. Compatibilité. Trois conditions doivent être remplies pour empiler deux résultats. Nommez-les, et dites ce qui décide du nom des colonnes du résultat final.
17. Pourquoi `UNION ALL` doit-il être votre choix par défaut, et dans quel cas précis passe-t-on à `UNION` ?
1. `LATERAL`. Que permet-il qu'une sous-requête ordinaire du `FROM` ne permet pas ?
2. Dans `LEFT JOIN LATERAL (…) AS t ON true`, à quoi sert le `ON true`, et à quoi sert le `LEFT` ?
3. Le top N par groupe s'écrit de trois façons. Citez-les et donnez, pour chacune, le cas où elle est le meilleur choix.
4. Le rapprochement. Pourquoi compare-t-on une copie à sa source dans les deux sens, et pourquoi en `EXCEPT ALL` plutôt qu'en `EXCEPT` ?
5. `[OLTP]` L'anti-jointure qui n'en est pas une. Que rend `FROM production.product ANTI JOIN sales.salesorderdetail USING (productid)` sur PostgreSQL, et pourquoi sans erreur ?

Questions pratiques

P1. `[OLTP]` Les produits jamais commandés. Écrivez la requête qui liste le nom et le prix de liste des produits de `production.product` qui n'apparaissent dans aucune ligne de `sales.salesorderdetail`. Utilisez l'écriture que la théorie recommande.

© 2026 iAlexMG. Tous droits réservés.

P2. [OLAP] Les clients sans commande récente. Sur l'entrepôt, listez les `CustomerKey` de `DimCustomer` qui n'ont aucune ligne dans `FactInternetSales` dont le `OrderDateKey` dépasse 20130101. Attention au piège : il ne s'agit pas des clients sans aucune commande.

P3. [OLTP] Le prix relatif. Pour chaque produit ayant une sous-catégorie, affichez son nom, son prix, le prix moyen de sa sous-catégorie, et l'écart entre les deux. Écrivez-la deux fois, une fois avec une sous-requête corrélée, une fois avec une fonction de fenêtrage, puis dites laquelle vous gardez et pourquoi.

P4. [OLAP] Les deux catégories. Listez les `CustomerKey` qui ont acheté au moins un produit de la catégorie `Bikes` et au moins un de la catégorie `Clothing`, en utilisant un opérateur d'ensemble. Puis écrivez la même chose sans opérateur d'ensemble, et comparez la lisibilité.

P5. [OLTP] Le top 2 par sous-catégorie. Affichez, pour chaque sous-catégorie, ses deux produits les plus chers, en conservant les sous-catégories qui n'ont aucun produit. Utilisez `LATERAL`.

P6. [OLAP] Le même top 2, en `QUALIFY`. Écrivez la version la plus courte possible, puis dites en une phrase ce que la version `LATERAL` fait que celle-ci ne fait pas.

P7. Le contrôle de cohérence. Écrivez, sur le terrain de votre choix, la requête qui compte les lignes de fait dont la clé de dimension n'a aucune correspondance. Puis transformez-la pour qu'elle renvoie non pas un compte, mais les dix clés fautives les plus fréquentes.

P8. [OLTP] L'anti-jointure sur colonne nullable. Listez les sous-catégories de `production.productsubcategory` auxquelles aucun produit n'est rattaché. Écrivez d'abord la version `NOT IN`, constatez le résultat, puis corrigez-la de deux façons différentes. (avancé)

P9. [OLAP] Les clients fidèles. Comptez les clients qui ont acheté en ligne en 2012 et en 2013, puis vérifiez que ce compte se tient avec les 287 perdus et les 14 461 gagnés du labo.

P10. [OLTP] Le rapprochement complet. Prenez comme source les lignes de commande des commandes 50000 à 50100. Fabriquez une copie qui perd la dernière ligne et où la première a gagné un cent. Rapprochez les deux en `EXCEPT ALL`, dans les deux sens, et expliquez chaque nombre. (avancé)

Diagnostics

E1. Un analyste veut les produits jamais vendus. Il écrit :

```
SELECT p.name
FROM   production.product AS p
WHERE  p.productid NOT IN (SELECT d.productid
                           FROM   sales.salesorderdetail AS d
                           WHERE  d.orderqty > 5);
```

Il obtient un résultat, mais il est faux. Deux défauts se cumulent ici. Trouvez-les et corrigez.

E2. Une requête censée afficher le nombre de commandes par client :

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT c.customerid,
       (SELECT COUNT(*) FROM sales.salesorderheader) AS nb_commandes
FROM   sales.customer AS c;
```

Toutes les lignes affichent le même nombre. Expliquez pourquoi et corrigez.

E3. Un collègue empile deux résultats :

```sql
SELECT productid, name       FROM production.product WHERE listprice > 1000
UNION
SELECT name,      productid  FROM production.product WHERE listprice = 0;
```

La requête échoue sur PostgreSQL. Elle aurait pu ne pas échouer. Expliquez ce qui se passe, et dites dans quel cas le même genre d'erreur passerait inaperçu.

E4. Une requête de contrôle qui ne contrôle rien :

```sql
SELECT COUNT(*)
FROM   FactInternetSales AS f
WHERE  f.ProductKey NOT IN (SELECT ProductKey FROM DimProduct WHERE Status = 'Current');
```

Elle renvoie 0 et l'auteur conclut que tout va bien. Donnez deux raisons distinctes pour lesquelles ce zéro ne prouve rien.

E5. Le top N qui perd des groupes :

```sql
SELECT e.businessentityid, top.salesorderid, top.totaldue
FROM   humanresources.employee AS e,
       LATERAL (SELECT h.salesorderid, h.totaldue FROM sales.salesorderheader AS h
                WHERE h.salespersonid = e.businessentityid
                ORDER BY h.totaldue DESC LIMIT 3) AS top;
```

Le rapport devait couvrir les 290 employés, et n'en montre que 17. Expliquez le mécanisme, corrigez, et dites pourquoi la disparition passe si facilement inaperçue dans un rapport livré. (avancé)

E6. Après chaque rechargement, un contrôle vérifie que la table copiée est identique à sa source :

```sql
SELECT COUNT(*) AS ecarts
FROM (SELECT * FROM copie EXCEPT SELECT * FROM source) AS x;
```

Il répond zéro tous les soirs. Un matin, les ventes de la copie dépassent celles de la source. Dites tout ce que ce contrôle ne voit pas, et corrigez-le.

Questions à développer

© 2026 iAlexMG. Tous droits réservés.

A1. La règle de la section 1 dit qu'une sous-requête doit répondre à une question qu'une jointure ne sait pas poser. Défendez cette règle en un paragraphe, puis donnez un contre-exemple : un cas où la sous-requête est préférable alors qu'une jointure suffirait techniquement.

A2. On vous dit : « nous avons remplacé tous les `NOT IN` par des `NOT EXISTS` dans notre dépôt, la migration a pris deux jours ». Expliquez en un paragraphe pourquoi ce travail se justifie même si aucune requête ne renvoyait de résultat faux au moment de la migration.

A3. Comparez `EXCEPT` et `NOT EXISTS` comme façons d'exprimer une différence. Décrivez précisément ce que chacun compare, et donnez un cas où remplacer l'un par l'autre change le résultat.

A4. Un architecte affirme que `LATERAL` est un pis-aller à éviter, parce qu'il force une exécution pour chaque ligne de la table de gauche. Répondez-lui en un paragraphe. Votre réponse doit reconnaître ce qu'il y a de juste dans son objection. (avancé)

Mises en situation

M1. Coopérative maraîchère Deux-Rives, la relance commerciale. La coopérative vend des paniers hebdomadaires à quatre mille foyers et tient trois tables : `abonne`, `livraison` et `ligne_livraison`. Le service commercial veut trois listes pour sa campagne d'automne : les abonnés qui n'ont reçu aucune livraison depuis trois mois ; ceux qui prenaient des produits laitiers l'an dernier et n'en prennent plus ; et, pour chaque région, les cinq abonnés au plus fort volume annuel.

Décrivez la requête de chaque liste, en nommant l'outil du module que vous employez et pourquoi. Pour la deuxième, dites précisément où le piège `NULL` pourrait vous atteindre. Vous n'avez pas besoin d'écrire le SQL complet, mais votre réponse doit être assez précise pour qu'un collègue puisse le faire.

M2. Clinique vétérinaire Saint-Élie, l'audit du référentiel. La clinique a migré son logiciel de gestion. L'ancien système et le nouveau ont chacun exporté leurs tables `animal`, `proprietaire` et `consultation` en CSV, chargées telles quelles dans un entrepôt. La directrice veut savoir si la migration a perdu quelque chose.

Proposez une batterie de contrôles fondée sur les outils du module. Dites, pour chaque contrôle, quel outil vous employez et ce qu'un résultat non vide signifierait. Précisez ensuite ce qu'un résultat vide **ne** prouve **pas**, et quel contrôle supplémentaire il faudrait pour combler ce trou.

M3. Plateforme de location d'outillage Ferblanc, le tableau de bord des sous-utilisés. La plateforme gère huit cents machines louées à la journée, avec les tables `machine`, `agence`, `location` et `categorie`. La direction veut un tableau mensuel qui montre, par agence, les machines jamais louées du mois, les trois machines les plus rentables, et les catégories présentes dans une agence mais absentes d'une autre.

Concevez la requête ou l'ensemble de requêtes. Justifiez le choix entre `LATERAL` et une fonction de fenêtrage pour la deuxième demande, en distinguant ce que vous feriez sur chacun des deux terrains de la formation. Dites enfin quelle partie de ce tableau de bord vous refuseriez de livrer en l'état, et quelle question vous poseriez à la direction avant d'aller plus loin. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés.

Questions courtes

1. Par exemple « quels produits n'ont jamais été vendus ? » et « quels clients ont acheté A mais pas B ? ». Ce qui leur manque : elles portent sur une absence ou sur une comparaison entre deux ensembles, alors qu'une agrégation ne sait décrire que ce qui est présent dans la table qu'elle parcourt.

Théorie : `Theorie_07_SQL.md`, section 1.

2. La sous-requête scalaire renvoie une seule valeur et vit partout où une valeur est attendue (`SELECT`, `WHERE`, `HAVING`). La sous-requête de table renvoie des lignes et des colonnes et vit dans le `FROM`, avec un alias obligatoire. La sous-requête de liste renvoie une colonne et plusieurs lignes, et s'emploie avec `IN`, `ANY`, `ALL` ou `EXISTS`.

Théorie : `Theorie_07_SQL.md`, section 2.

3. Elle vaut `NULL`. La comparaison avec `NULL` ne vaut ni vrai ni faux mais inconnu, donc la ligne ne passe pas le filtre et disparaît du résultat, sans erreur ni avertissement.

Théorie : `Theorie_07_SQL.md`, section 2.

4. Le moteur lève une erreur à l'exécution. C'est moins dangereux parce qu'une erreur se voit : elle arrête le traitement et vous force à corriger. Le cas de la question 3 produit un résultat plausible mais faux, que personne ne remarque.

Théorie : `Theorie_07_SQL.md`, section 2.

5. Elle référence une colonne d'une table déclarée dans la requête englobante, pas dans la sous-requête elle-même. Concrètement, on voit un alias qui n'est défini nulle part à l'intérieur des parenthèses.

Théorie : `Theorie_07_SQL.md`, section 3.

6. Parce que les moteurs modernes décorrèlent une partie de ces requêtes : un `EXISTS` ou un `IN` corrélé placé dans le `WHERE` devient une semi-jointure avant l'exécution, sans boucle. La règle n'est pas générale pour autant. Une sous-requête scalaire corrélée placée dans le `SELECT` reste un `SubPlan`, relancé pour chaque ligne, et c'est au rédacteur de la réécrire en jointure agrégée s'il veut autre chose. D'où la vraie réponse : ni croire le mythe, ni croire son contraire, mais lire le plan d'exécution et mesurer. La lenteur se constate, elle ne se devine pas.

Théorie : `Theorie_07_SQL.md`, section 3.

7. Non, le résultat serait identique : `EXISTS` ne regarde jamais les colonnes renvoyées, seulement s'il existe au moins une ligne. La convention `SELECT 1` sert à dire au lecteur qu'on ne s'intéresse pas au contenu de la sous-requête.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_07_SQL.md`, section 4.

8. La semi-jointure filtre la table du `FROM` en vérifiant qu'au moins une correspondance existe dans une autre table, sans rien ramener de celle-ci : c'est `EXISTS`. L'anti-jointure garde au contraire les lignes de la table du `FROM` qui n'ont aucune correspondance : c'est `NOT EXISTS`.

Théorie : `Theorie_07_SQL.md`, section 4.

9. Parce que `EXISTS` s'arrête dès la première ligne trouvée, alors que `COUNT(*)` doit parcourir toutes les lignes correspondantes avant de pouvoir comparer le total à zéro.

Théorie : `Theorie_07_SQL.md`, section 4.

10. `x NOT IN (a, b, NULL)` se développe en `x <> a AND x <> b AND x <> NULL`. Le dernier terme vaut inconnu, jamais vrai. Or un `AND` dont un terme est inconnu ne peut plus valoir vrai : au mieux il vaut inconnu. Aucune ligne ne satisfait donc le filtre.

Théorie : `Theorie_07_SQL.md`, section 5.

11. Parce qu'une erreur de syntaxe arrête la requête et se corrige. Ici, la requête s'exécute et renvoie zéro ligne, ce qui est une réponse crédible à « qu'est-ce qui manque ». Le résultat faux part en production sans que rien ne le signale.

Théorie : `Theorie_07_SQL.md`, section 5.

12. `NOT EXISTS`, `NOT IN`, et le `LEFT JOIN` suivi d'un test `IS NULL` sur la clé de la table de droite. Par défaut on écrit `NOT EXISTS` : c'est la seule des trois qui soit à la fois correcte quelles que soient les valeurs et lisible dans son intention.

Théorie : `Theorie_07_SQL.md`, sections 4 et 5.

13. `x IN (…)` équivaut à `x = ANY (…)`. `x NOT IN (…)` équivaut à `x <> ALL (…)`.

Théorie : `Theorie_07_SQL.md`, section 5.

14. `x > ALL (…)` sur un ensemble vide vaut vrai, parce qu'aucun élément ne vient contredire l'affirmation. `x > ANY (…)` vaut faux, parce qu'aucun élément ne vient la confirmer. La conséquence pratique : une requête bâtie sur `ALL` laisse passer toutes les lignes le jour où sa sous-requête ne renvoie plus rien.

Théorie : `Theorie_07_SQL.md`, section 5.

15. `UNION`, `UNION ALL`, `INTERSECT`, `EXCEPT`. Seul `UNION ALL` ne dédoublonne pas, parmi les quatre ; `INTERSECT` et `EXCEPT` ont aussi une version `ALL`, qui compte les répétitions, sur nos deux terrains.

Théorie : `Theorie_07_SQL.md`, section 6.

16. Même nombre de colonnes, types compatibles colonne par colonne, et l'ordre des colonnes fait foi. Ce sont les noms du premier `SELECT` qui nomment le résultat ; les noms des suivants sont ignorés.

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_07_SQL.md`, section 6.

17. Parce que le dédoublonnage oblige le moteur à trier ou à hacher tout le résultat, ce qui est souvent l'opération la plus chère de la requête, et parce qu'on n'a le plus souvent aucun doublon à supprimer. On passe à `UNION` uniquement quand on sait qu'il y a des doublons et qu'on veut vraiment s'en débarrasser.

Théorie : `Theorie_07_SQL.md`, section 6.

18. Il autorise la sous-requête à référencer les tables qui la précèdent dans la clause `FROM`. Sans lui, une sous-requête du `FROM` est évaluée dans le vide, avant les autres tables, et ne peut donc rien en lire.

Théorie : `Theorie_07_SQL.md`, section 7.

19. Le `ON true` est une formalité de syntaxe : la condition de rapprochement vit déjà dans le `WHERE` de la sous-requête, il ne reste rien à mettre dans le `ON`, mais la grammaire d'un `JOIN` en exige un. Le `LEFT` conserve les lignes de la table de gauche pour lesquelles la sous-requête ne renvoie rien ; sans lui, elles disparaissent du résultat.

Théorie : `Theorie_07_SQL.md`, section 7.

20. La fonction de fenêtrage avec `ROW_NUMBER`, puis filtrage du rang : le meilleur choix quand il s'agit vraiment d'un classement, parce qu'elle ne fait qu'un passage. `QUALIFY` sur l'entrepôt : la même chose en plus court, quand on est sur cette plateforme. `LATERAL` : le meilleur choix quand la sous-requête fait autre chose qu'un classement, par exemple aller chercher la dernière valeur connue avant une date.

Théorie : `Theorie_07_SQL.md`, section 7.

21. Parce qu'`EXCEPT` ne rend que ce que le premier ensemble a de plus : une copie à laquelle il manque des lignes passe le test à zéro si l'on ne compare que dans un sens. Et parce qu'`EXCEPT` dédoublonne : une copie qui a reçu deux fois une partie de sa source a les mêmes valeurs, et il conclut à l'égalité. `EXCEPT ALL` compte les occurrences et voit les lignes en trop. Zéro dans les deux sens, en `EXCEPT ALL`, et seulement là, les deux tables portent les mêmes lignes.

Théorie : `Theorie_07_SQL.md`, section 6.

22. `[OLTP]` Les 121 317 lignes de commande, soit une jointure interne, exactement le contraire de l'anti-jointure voulue. PostgreSQL ne connaît pas `ANTI JOIN` ; sans alias après la table, il lit `ANTI` comme l'alias de `production.product`, puis `JOIN … USING` comme une jointure ordinaire. Avec un alias déjà posé, `AS p ANTI JOIN`, la même écriture lève une erreur de syntaxe.

Théorie : `Theorie_07_SQL.md`, section 4.

Questions pratiques

P1. `[OLTP]`

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT p.name, p.listprice
FROM   production.product AS p
WHERE  NOT EXISTS (SELECT 1
                   FROM   sales.salesorderdetail AS d
                   WHERE  d.productid = p.productid)
ORDER  BY p.name;
```

Environ deux cent trente-huit lignes. `NOT EXISTS` plutôt que `NOT IN` : même si `salesorderdetail.productid` n'est pas nullable aujourd'hui, la règle ne dépend pas d'une vérification qu'il faudrait refaire à chaque évolution du schéma.

Théorie : `Theorie_07_SQL.md`, section 4.

P2. `[OLAP]` Le piège de l'énoncé est qu'un client sans aucune commande satisfait aussi la condition. Si l'on veut les clients qui ont commandé mais plus depuis 2013, il faut l'exiger explicitement.

```sql
SELECT c.CustomerKey
FROM   DimCustomer AS c
WHERE  EXISTS     (SELECT 1 FROM FactInternetSales AS f
                   WHERE f.CustomerKey = c.CustomerKey)
  AND  NOT EXISTS (SELECT 1 FROM FactInternetSales AS f
                   WHERE f.CustomerKey = c.CustomerKey AND f.OrderDateKey > 20130101);
```

Sans le premier `EXISTS`, la liste mélange les clients inactifs et les clients qui n'ont jamais rien acheté, deux populations qui n'appellent pas la même action commerciale.

Théorie : `Theorie_07_SQL.md`, sections 4 et 5.

P3. `[OLTP]`

```sql
-- Version corrélée.
SELECT p.name, p.listprice,
       (SELECT AVG(q.listprice) FROM production.product AS q
        WHERE q.productsubcategoryid = p.productsubcategoryid) AS moyenne_sc,
       p.listprice - (SELECT AVG(q.listprice) FROM production.product AS q
                      WHERE q.productsubcategoryid = p.productsubcategoryid) AS ecart
FROM   production.product AS p
WHERE  p.productsubcategoryid IS NOT NULL;

-- Version fenêtre.
SELECT p.name, p.listprice,
       AVG(p.listprice) OVER (PARTITION BY p.productsubcategoryid) AS moyenne_sc,
       p.listprice - AVG(p.listprice) OVER (PARTITION BY p.productsubcategoryid) AS ecart
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   production.product AS p
WHERE  p.productsubcategoryid IS NOT NULL;
```

On garde la fenêtre. Elle exprime la même chose en un seul passage, elle ne répète pas le calcul deux fois, et elle reste juste si l'on ajoute une colonne. La version corrélée a un mérite pédagogique et rien d'autre ici.

Théorie : `Theorie_07_SQL.md`, section 3.

P4. `[OLAP]`

```sql
WITH achats AS (
    SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie
    FROM   FactInternetSales     AS f
    JOIN   DimProduct            AS p  ON p.ProductKey = f.ProductKey
    JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
    JOIN   DimProductCategory    AS c  ON c.ProductCategoryKey = sc.ProductCategoryKey
)
SELECT CustomerKey FROM achats WHERE categorie = 'Bikes'
INTERSECT
SELECT CustomerKey FROM achats WHERE categorie = 'Clothing';
```

Sans opérateur d'ensemble, avec deux `EXISTS` sur la même table de faits. La CTE ne vit que le temps de la requête qui la déclare : la seconde écriture doit donc la déclarer à nouveau, sans quoi DuckDB répond `Table with name achats does not exist`, mesuré.

```sql
WITH achats AS (
    SELECT f.CustomerKey, c.EnglishProductCategoryName AS categorie
    FROM   FactInternetSales     AS f
    JOIN   DimProduct            AS p  ON p.ProductKey = f.ProductKey
    JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
    JOIN   DimProductCategory    AS c  ON c.ProductCategoryKey = sc.ProductCategoryKey
)
SELECT c.CustomerKey
FROM   DimCustomer AS c
WHERE  EXISTS (SELECT 1 FROM achats AS a
               WHERE a.CustomerKey = c.CustomerKey AND a.categorie = 'Bikes')
  AND  EXISTS (SELECT 1 FROM achats AS a
               WHERE a.CustomerKey = c.CustomerKey AND a.categorie = 'Clothing');
```

Les deux rendent les mêmes 3 172 clients. La seconde se lit mieux dès qu'on veut ajouter une troisième condition, et elle permet de garder d'autres colonnes de `DimCustomer`. L'`INTERSECT` est plus concis mais ne renvoie que la clé.

Théorie : `Theorie_07_SQL.md`, section 6.

P5. `[OLTP]`

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT sc.name AS sous_categorie, top.name, top.listprice
FROM   production.productsubcategory AS sc
LEFT   JOIN LATERAL (SELECT p.name, p.listprice
                     FROM   production.product AS p
                     WHERE  p.productsubcategoryid = sc.productsubcategoryid
                     ORDER  BY p.listprice DESC
                     LIMIT  2) AS top ON true
ORDER  BY sous_categorie, top.listprice DESC;
```

Le `LEFT` est ce qui répond à « en conservant les sous-catégories qui n'ont aucun produit ». Les lignes correspondantes porteraient alors des `NULL` dans les colonnes de `top`.

Au conditionnel, parce que sur cette base il n'y en a aucune : les 37 sous-catégories ont toutes au moins un produit, et la requête rend 37 groupes avec ou sans le `LEFT`. C'est exactement la situation qui fait tomber les gens, et elle mérite d'être vue ici plutôt que dans six mois. Une requête dont le `LEFT` ne change rien aujourd'hui n'est pas une requête où le `LEFT` est inutile : c'est une requête où la donnée ne l'a pas encore exercé. Le jour où une sous-catégorie naît sans produit, la version sans `LEFT` la perd en silence. Écrivez donc le `LEFT` d'après la question posée, jamais d'après ce que les données contiennent au moment où vous écrivez. Le diagnostic E5 montre le même mécanisme sur un terrain où la perte est immédiate, 17 groupes au lieu de 290.

Théorie : `Theorie_07_SQL.md`, section 7.

P6. `[OLAP]`

```sql
SELECT sc.EnglishProductSubcategoryName AS sous_categorie,
       p.EnglishProductName, p.ListPrice
FROM   DimProduct AS p
JOIN   DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
QUALIFY ROW_NUMBER() OVER (PARTITION BY sc.ProductSubcategoryKey
                           ORDER BY p.ListPrice DESC) <= 2;
```

Ce que la version `LATERAL` fait de plus : elle part des sous-catégories, donc elle peut conserver celles qui n'ont aucun produit. Celle-ci part des produits et ne peut pas les inventer.

Théorie : `Theorie_07_SQL.md`, sections 7 et 8.

P7. Sur l'entrepôt, le compte puis le détail :

```sql
-- Le compte.
SELECT COUNT(*) AS lignes_orphelines
FROM   FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey);

-- Les dix clés fautives les plus fréquentes.
SELECT f.ProductKey, COUNT(*) AS nb_lignes
```


© 2026 iAlexMG. Tous droits réservés.

```sql
FROM   FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM DimProduct AS p WHERE p.ProductKey = f.ProductKey)
GROUP  BY f.ProductKey
ORDER  BY nb_lignes DESC
LIMIT  10;
```

Le passage du compte au détail est ce qui rend le contrôle exploitable : un nombre dit qu'il y a un problème, la liste des clés dit par où commencer.

Théorie : `Theorie_07_SQL.md`, section 4.

P8. [OLTP]

```sql
-- Version fautive : renvoie 0.
SELECT COUNT(*)
FROM   production.productsubcategory AS sc
WHERE  sc.productsubcategoryid NOT IN (SELECT p.productsubcategoryid
                                       FROM   production.product AS p);

-- Correction 1 : écarter les NULL de la sous-requête.
SELECT COUNT(*)
FROM   production.productsubcategory AS sc
WHERE  sc.productsubcategoryid NOT IN (SELECT p.productsubcategoryid
                                       FROM   production.product AS p
                                       WHERE  p.productsubcategoryid IS NOT NULL);

-- Correction 2 : la bonne, celle qui ne demande rien à savoir sur les données.
SELECT COUNT(*)
FROM   production.productsubcategory AS sc
WHERE  NOT EXISTS (SELECT 1 FROM production.product AS p
                   WHERE  p.productsubcategoryid = sc.productsubcategoryid);
```

Les deux corrections donnent le même compte. La première exige de savoir que la colonne est nullable, ce qu'il faudrait revérifier à chaque évolution du schéma. La seconde est juste par construction. C'est pour cela que la règle est `NOT EXISTS`, et non « `NOT IN` avec précaution ».

Théorie : `Theorie_07_SQL.md`, section 5.

P9. [OLAP]

```sql
WITH clients AS (
    SELECT DISTINCT CustomerKey, YEAR(OrderDate) AS annee
    FROM   FactInternetSales
)
SELECT COUNT(*) AS fideles
FROM (SELECT CustomerKey FROM clients WHERE annee = 2012
```


© 2026 iAlexMG. Tous droits réservés.

```
INTERSECT
SELECT CustomerKey FROM clients WHERE annee = 2013);
```

2 968 clients. Le compte se tient des deux côtés : 3 255 clients de 2012 moins 287 perdus, et 17 429 clients de 2013 moins 14 461 gagnés, font tous deux 2 968. Une vérification de ce genre, où trois requêtes indépendantes doivent retomber sur le même chiffre, vaut mieux qu'une relecture.

Théorie : `Theorie_07_SQL.md`, section 6.

P10. `[OLTP]`

```
WITH source AS (
    SELECT salesorderid, salesorderdetailid, linetotal
    FROM   sales.salesorderdetail
    WHERE  salesorderid BETWEEN 50000 AND 50100
),
cible AS (
    SELECT salesorderid, salesorderdetailid,
           CASE WHEN salesorderdetailid = (SELECT MIN(salesorderdetailid) FROM source)
                THEN linetotal + 0.01 ELSE linetotal END AS linetotal
    FROM   source
    WHERE  salesorderdetailid <> (SELECT MAX(salesorderdetailid) FROM source)
)
SELECT (SELECT COUNT(*) FROM source) AS lignes_source,
       (SELECT COUNT(*) FROM cible)  AS lignes_cible,
       (SELECT COUNT(*) FROM (SELECT * FROM cible  EXCEPT ALL SELECT * FROM source) AS x) AS cible_sans_source,
       (SELECT COUNT(*) FROM (SELECT * FROM source EXCEPT ALL SELECT * FROM cible)  AS x) AS source_sans_cible;
```

101 lignes à la source, 100 à la copie, 1 ligne de la copie absente de la source, 2 lignes de la source absentes de la copie. La ligne modifiée compte des deux côtés : sa nouvelle version n'existe pas à la source, son ancienne version n'existe plus dans la copie. La ligne perdue ne compte que du côté de la source. Un rapprochement ne dit donc pas « deux erreurs » mais « une modification et une perte », et c'est la clé (`salesorderdetailid`) qui permet de les séparer : une clé présente des deux côtés avec deux valeurs différentes est une modification, une clé d'un seul côté une perte ou un ajout.

Théorie : `Theorie_07_SQL.md`, section 6.

Diagnostics

E1. Deux défauts se cumulent.

Le premier est le `NOT IN` sur une sous-requête, avec le risque `NULL` habituel. Le second est plus subtil et c'est lui qui rend le résultat faux même sans aucun `NULL` : le filtre `orderqty > 5` restreint la liste des produits vendus aux seules lignes de plus de cinq unités. Un produit vendu une seule fois, en deux exemplaires, sera donc déclaré « jamais vendu ». La condition n'a rien à faire dans une anti-jointure qui teste l'existence.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT p.name
FROM   production.product AS p
WHERE  NOT EXISTS (SELECT 1 FROM sales.salesorderdetail AS d
                   WHERE  d.productid = p.productid);
```

La leçon générale : dans une anti-jointure, toute condition ajoutée à la sous-requête rétrécit l'ensemble de référence, donc élargit le résultat. Il faut se demander à chaque fois si c'est voulu.

Théorie : `Theorie_07_SQL.md`, sections 4 et 5.

E2. La sous-requête n'est pas corrélée : rien à l'intérieur ne fait référence à `c`. Elle est donc calculée une fois et la même valeur, le total général des commandes, s'affiche sur chaque ligne.

```sql
SELECT c.customerid,
       (SELECT COUNT(*) FROM sales.salesorderheader AS h
        WHERE  h.customerid = c.customerid) AS nb_commandes
FROM   sales.customer AS c;
```

Théorie : `Theorie_07_SQL.md`, section 3.

E3. Les types ne correspondent pas colonne par colonne : le premier `SELECT` empile un entier puis un texte, le second un texte puis un entier. PostgreSQL refuse.

Le même genre d'erreur passerait inaperçu si les deux colonnes inversées avaient des types compatibles, par exemple deux textes ou deux entiers. La requête s'exécuterait alors sans un mot, en empilant des valeurs qui n'ont rien à voir sous le nom de colonne du premier `SELECT`. Mesuré sur l'entrepôt avec deux entiers, la clé d'un produit et son stock de sécurité inversés : 210 et 500 d'un côté, 500 et 210 de l'autre, sans erreur. C'est pourquoi il faut lire l'ordre des colonnes plutôt que se fier à leurs noms.

Une bonne réponse nomme la parade qui existe sur l'entrepôt : `UNION BY NAME`, qui aligne les colonnes par leur nom. Écrite ainsi, la requête du collègue passerait sur DuckDB et rangerait chaque valeur à sa place, mesuré ; PostgreSQL ne connaît pas cette forme et la refuse par une erreur de syntaxe. Snowflake l'a aussi, d'après sa documentation.

Théorie : `Theorie_07_SQL.md`, section 6.

E4. Deux raisons distinctes.

La première tient au `NOT IN` : si `DimProduct.ProductKey` contenait un `NULL`, le résultat serait zéro quelles que soient les données, donc le zéro ne prouve rien.

La seconde tient au filtre `Status = 'Current'`. La sous-requête ne liste que les versions courantes de la dimension. Une ligne de fait pointant une version historique parfaitement valide serait comptée comme orpheline, ou l'inverse selon l'écriture. Le contrôle ne mesure donc pas ce que son auteur croit mesurer.

Un contrôle correct porte sur la dimension entière et s'écrit avec `NOT EXISTS`.

Théorie : `Theorie_07_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

E5. La virgule et `LATERAL` produisent une jointure interne : un employé dont la sous-requête ne renvoie aucune ligne disparaît du résultat. Les 273 employés absents n'ont jamais pris de commande, ce qui est la situation de la grande majorité d'entre eux : sur 290 employés, 17 seulement sont vendeurs.

```
SELECT e.businessentityid, top.salesorderid, top.totaldue
FROM   humanresources.employee AS e
LEFT   JOIN LATERAL (SELECT h.salesorderid, h.totaldue FROM sales.salesorderheader AS h
                     WHERE h.salespersonid = e.businessentityid
                     ORDER BY h.totaldue DESC LIMIT 3) AS top ON true;
```

Attendu : 290 employés couverts au lieu de 17, les 273 autres apparaissant avec des `NULL` dans les colonnes de commande.

Reste la question qui compte, celle de la disparition inaperçue. Un rapport de top N ne montre que des lignes pleines : il n'y a pas de trou visible, pas de cellule vide, rien qui signale l'absence. Le lecteur voit dix-sept employés avec leurs meilleures commandes, et rien ne lui dit qu'il en manque 273. Le seul contrôle qui l'aurait attrapé est celui qu'on oublie toujours : compter les groupes en entrée et les groupes en sortie, et vérifier qu'ils coïncident. C'est le même geste que le contrôle d'exactitude du Cours 12.

C'est un défaut typique d'un rapport de pilotage : les groupes vides sont précisément ceux dont la direction voudrait entendre parler, et ce sont eux qui s'effacent.

Théorie : `Theorie_07_SQL.md`, section 7.

E6. Le contrôle ne regarde que dans un sens, et il dédoublonne. Il voit une ligne de la copie que la source n'a pas, et rien d'autre. Il ne voit pas une ligne perdue en route, qui n'existe que dans la source. Il ne voit pas non plus une ligne chargée deux fois, puisque `EXCEPT` compare des valeurs et que la copie n'en a aucune de nouvelle : c'est le cas de ce matin, des ventes en double dont la somme dépasse celle de la source. La correction compare en `EXCEPT ALL`, dans les deux sens :

```
SELECT (SELECT COUNT(*) FROM (SELECT * FROM copie  EXCEPT ALL SELECT * FROM source) AS x) AS en_trop,
       (SELECT COUNT(*) FROM (SELECT * FROM source EXCEPT ALL SELECT * FROM copie)  AS x) AS manquantes;
```

Le contrôle passe quand les deux valent zéro. Les comptes de lignes des deux tables, posés à côté, coûtent une requête de plus et attrapent le double chargement d'un coup d'œil.

Théorie : `Theorie_07_SQL.md`, section 6.

Questions à développer

A1. Une bonne réponse défend la règle par le coût de lecture. Une jointure déclare une relation entre deux tables, elle se lit de gauche à droite et le moteur la traite d'un bloc. Une sous-requête introduit un niveau d'imbrication qui se lit de l'intérieur vers l'extérieur, à contresens du sens de lecture naturel. À résultat égal, la jointure demande donc moins d'effort au lecteur suivant, et c'est ce lecteur qui paiera vos choix.

© 2026 iAlexMG. Tous droits réservés.

Le contre-exemple attendu : la semi-jointure. `EXISTS` exprime « au moins une correspondance » sans risquer de multiplier les lignes de la table de gauche, là où une jointure sur une relation un-vers-plusieurs duplique les lignes et oblige à un `DISTINCT` correctif. Le `DISTINCT` masque alors l'intention et coûte un dédoublonnage. On accepte aussi comme contre-exemple la sous-requête scalaire de contexte quand la valeur est vraiment globale.

Est également acceptable une réponse qui conteste la règle en soulignant qu'elle privilégie la lisibilité sur la performance, à condition de dire pourquoi ce choix est le bon par défaut.

Théorie : `Theorie_07_SQL.md`, section 1.

A2. Le point à faire passer : le fait qu'aucune requête ne soit fausse aujourd'hui est une propriété des **données**, pas du code. Elle tient tant qu'aucune colonne concernée ne reçoit de `NULL`. Or c'est exactement le genre de propriété qui change sans prévenir, quand une colonne devient optionnelle, quand une source ajoute des lignes incomplètes, ou quand un `LEFT JOIN` en amont introduit des `NULL` là où il n'y en avait pas.

Une bonne réponse ajoute que le mode de défaillance est silencieux : la requête ne casse pas, elle renvoie zéro ligne. Le jour où le problème arrive, il ne se manifeste pas par une alerte mais par un rapport qui semble dire que tout va bien. Deux jours de migration achètent la disparition d'une classe entière de pannes invisibles, ce qui est un excellent rapport.

Théorie : `Theorie_07_SQL.md`, section 5.

A3. `NOT EXISTS` compare selon une **condition** que vous écrivez, généralement une égalité de clé, et il filtre les lignes de la table du `FROM` en conservant toutes leurs colonnes. `EXCEPT` compare des **lignes entières**, colonne par colonne et dans l'ordre, entre deux résultats de même forme, et il dédoublonne le résultat.

Le cas où le remplacement change le résultat : dès que les deux côtés n'ont pas la même liste de colonnes. Si l'on ajoute une colonne au premier `SELECT` d'un `EXCEPT`, deux lignes qui différaient seulement par cette colonne cessent d'être considérées comme identiques, et la différence d'ensembles change. Un `NOT EXISTS` sur la clé n'aurait pas bougé. Le dédoublonnage d'`EXCEPT` est un second point de divergence : si la table de départ (celle du `FROM` pour `NOT EXISTS`, celle du premier `SELECT` pour `EXCEPT`) contient des doublons légitimes, `EXCEPT` les réduit à un, `NOT EXISTS` les conserve tous.

Théorie : `Theorie_07_SQL.md`, sections 4 et 6.

A4. L'objection est partiellement fondée et il faut le dire d'abord. `LATERAL` exprime bien une exécution pour chaque ligne de la table de gauche, et si cette table compte des millions de lignes pendant que la sous-requête parcourt une grosse table, le coût est réel et une fonction de fenêtrage fera mieux.

Mais l'objection généralise à tort, pour trois raisons. D'abord, `LATERAL` sert typiquement à partir d'une petite table de gauche, un référentiel de catégories ou d'agences, ce qui rend le nombre d'itérations dérisoire. Ensuite, le moteur optimise parfois ce motif ; le plan d'exécution le dit, et il faut le lire plutôt que le supposer, puisque la décorrélation ne joue pas partout, comme le labo l'a montré sur la scalaire du `SELECT`. Enfin, `LATERAL` exprime des choses qu'aucune fenêtre n'exprime : dès que la sous-requête fait autre chose qu'un classement, par exemple chercher la dernière valeur antérieure à une date, la fenêtre n'est plus une alternative.

Une bonne réponse conclut sur la méthode plutôt que sur le verdict : on écrit ce qui exprime la question, on lit le plan, et on ne remplace que si la mesure le justifie.

Théorie : `Theorie_07_SQL.md`, section 7.

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

M1. Réponse défendable.

La première liste est une anti-jointure avec condition de date : `NOT EXISTS` sur `livraison`, avec le filtre de date **dans** la sous-requête. Le point qui distingue une bonne réponse d'une réponse approximative est celui du diagnostic E1 : mettre le filtre de date dans le `WHERE` extérieur ne donnerait pas du tout la même chose. Il faut aussi décider si les abonnés n'ayant jamais reçu de livraison entrent dans la liste, et le dire.

La deuxième liste est une différence d'ensembles : les abonnés d'une période moins ceux d'une autre, avec `EXCEPT` ou avec un couple `EXISTS` et `NOT EXISTS`. Le piège `NULL` guette précisément si l'on écrit cette différence avec `NOT IN` sur une liste d'identifiants issue d'une jointure, où un `LEFT JOIN` en amont peut avoir introduit des `NULL` qui n'existaient dans aucune table de départ. C'est la réponse attendue à la question posée.

La troisième liste est un top N par groupe. Sur PostgreSQL, `LATERAL` ou fenêtre plus CTE ; sur l'entrepôt, `QUALIFY`. Une bonne réponse signale qu'il faut conserver les régions sans abonné actif, donc `LEFT JOIN LATERAL`, faute de quoi une région en difficulté disparaît du rapport, ce qui est l'inverse du besoin.

Théorie : `Theorie_07_SQL.md`, sections 4, 6 et 7.

M2. Une bonne batterie contient au moins trois contrôles.

Les entités perdues : `NOT EXISTS` de l'ancien vers le nouveau, sur la clé métier, pour chacune des trois tables. Un résultat non vide nomme ce qui a disparu. Les entités apparues : la même chose dans l'autre sens, qui révèle les doublons créés par la migration ou les lignes fabriquées. Les orphelins : `NOT EXISTS` de `consultation` vers `animal` et de `animal` vers `proprietaire`, dans le nouveau système, pour vérifier que les liens ont suivi.

La partie qui distingue une bonne réponse est la seconde. Un résultat vide ne prouve pas que rien n'a été perdu : il prouve seulement que chaque clé présente d'un côté l'est de l'autre. Il ne dit rien du **contenu** des lignes. Une migration peut conserver toutes les clés en tronquant les noms, en décalant les dates d'un fuseau, ou en arrondissant les montants. Le contrôle manquant est une comparaison colonne à colonne des lignes appariées, qu'`EXCEPT` fait très bien puisqu'il compare des lignes entières. C'est l'usage où `EXCEPT` bat `NOT EXISTS`, et le lien avec la question A3.

Théorie : `Theorie_07_SQL.md`, section 6.

M3. Réponse défendable.

Les machines jamais louées du mois sont une anti-jointure avec filtre de date dans la sous-requête, et il faut conserver les agences sans aucune location, donc partir de `agence` et non de `location`.

Pour les trois machines les plus rentables par agence, l'arbitrage attendu est le suivant. Sur l'entrepôt, `QUALIFY` sur une fenêtre : c'est un classement pur, la fenêtre est faite pour ça, et la table des locations est grosse. Sur PostgreSQL, la fenêtre plus CTE pour la même raison ; `LATERAL` ne se justifierait que si la table de gauche, les agences, est petite et si l'on veut conserver celles qui n'ont rien loué. Une bonne réponse mentionne ce dernier point, parce qu'une agence sans location est précisément une agence dont la direction veut entendre parler.

© 2026 iAlexMG. Tous droits réservés.

Les catégories présentes ici et absentes là sont un `EXCEPT` entre deux agences, ou une généralisation par anti-jointure si la comparaison doit se faire entre toutes les paires d'agences. Signaler que la version « toutes les paires » ne s'écrit pas avec `EXCEPT` est un bon point.

Ce qu'il faut refuser de livrer : la notion de machine « la plus rentable » n'est pas définie. Le chiffre d'affaires de location, la marge après entretien et le taux d'utilisation donnent trois classements différents, et une machine louée tous les jours à bas prix peut être plus rentable qu'une machine louée trois fois cher. La question à poser à la direction porte donc sur la définition de la rentabilité, et sur ce qu'elle compte faire du classement. Livrer un chiffre d'affaires en l'appelant rentabilité serait une faute que personne ne verrait avant qu'une décision soit prise dessus.

Théorie : `Theorie_07_SQL.md`, sections 4, 6 et 7.

© 2026 iAlexMG. Tous droits réservés.